מעיין

maayan

Course 2 — The Intelligence Layer: from lookup to reasoning

How the upgrade works, how to run and tune it, how to demo it, and how to reuse the pattern — a second self-study curriculum for the owner of this system.

maayan.ai

Modules 0–5 · 18 lessons · 2026-06-29

Contents

Module 0 — From lookup to reasoning (orientation)

- Lesson 0.1 — The ceiling we hit
- Lesson 0.2 — The three moves (the map)
- Lesson 0.3 — Feel the difference

Module 1 — Asking better (query intelligence)

- Lesson 1.1 — Why a verbatim query under-retrieves
- Lesson 1.2 — Multi-query & HyDE
- Lesson 1.3 — Lexicon-aware expansion (no model needed)
- Lesson 1.4 — Fusing the nets: RRF + the drop-in retriever

Module 2 — Thinking before answering (reasoning & synthesis)

- Lesson 2.1 — One pass vs. two stages
- Lesson 2.2 — The study map
- Lesson 2.3 — Synthesis: weave, don't list
- Lesson 2.4 — Checking yourself, and the rule that didn't change

Module 3 — Proving the lift (evaluation & cost)

- Lesson 3.1 — The cost ladder
- Lesson 3.2 — Measuring the lift with `eval-expand`

Module 4 — Tuning & playing (making it yours)

- Lesson 4.1 — The knobs, and recipes
- Lesson 4.2 — Editing the prompts (and diagnosing failures)

Module 5 — Demoing & explaining (telling the story)

- Lesson 5.1 — The five-minute demo
- Lesson 5.2 — Explaining it to three audiences
- Lesson 5.3 — The horizon: the reusable pattern

Module 0 — From lookup to reasoning (orientation)

See, in your own terminal, the difference between the old single-shot answer and the new reasoning answer — and hold a map of the three moves before we open each box.

Lesson 0.1 — The ceiling we hit

> Module 0, Lesson 1 · ~12 min read + a short hands-on. The one question this answers: **what does "it's just a fancy lookup system" actually mean in the code — and why does it cap how intelligent the answers can be?**

By the end of Course 1 you had something genuinely good: a system that retrieves the right passages, answers only from them, cites every claim, and refuses when it has no source. That is a faithful **lookup** system. This lesson is about the wall it hits — and why that wall is exactly where Course 2 begins.

What "single shot" means, concretely

Trace one `ask` as it worked before Prompt 31. It is three steps, each done **once**:

1. **Embed the question once.** The retriever calls `embed_query(question)` a single time and searches with that one vector. See `retrieve/retriever.py` — one query in, one ranked list out.
2. **Gate once.** The default-deny check compares the top relevance to `score_threshold`. If nothing clears it, refuse. (This part is sacred — we keep it untouched all course.)
3. **Generate once.** `generate/rag.py` builds one prompt — *here are the numbered sources, answer and cite* — and makes a single model call. Sources in, prose out.

```
question ──► embed once ──► search once ──► [gate] ──► one model call ──► answer
```

There is no step where the system *reconsiders* the question, *re-reads* the sources, or *checks* its own answer. It finds and it reports. That's lookup.

Why lookup hits a ceiling on Torah questions

Lookup works beautifully when the question shares words with the answer. It struggles exactly where chassidus lives: **conceptual** questions whose answer is phrased nothing like the ask.

- You ask about *the relationship between ביטול and רצוא ושוב*. The source that answers it may never use the word "relationship," may discuss the two ideas chapters apart, and may name them with abbreviations or synonyms. One embedding of your exact phrasing can sail right past it.

- Even when retrieval finds good passages, a single generation pass tends to **summarize each source in turn** rather than show how they *fit* — which agrees with which, which builds on which, where they're in tension. But "how the sources fit" *is* the substance of learning chassidus. Lookup gives you the quotes; it doesn't give you the *shiur*.
- And nothing checks whether each `[S#]` the model attached actually supports the sentence it's attached to. The grounding is enforced by *prompt*, not *proof*.

None of these are bugs. They're the ceiling of the single-shot shape. To go higher you change the *shape*: ask in more than one way, think before answering, and verify after. That's Modules 1, 2, and 2.4 respectively.

Hands-on

Pick a **conceptual** question from your own corpus — one whose answer you know lives in the text but is phrased differently from how you'd ask. (If you're unsure, something like `"מה הקשר בין צמצום לבריאת העולם"` over Tanya/Torah Ohr.)

1. See what one query retrieves. No backend needed — just retrieval:

```
uv run maayan search "מה הקשר בין צמצום לבריאת העולם" --k 8
```

Read the eight refs. Are the passages you'd *expect* a scholar to cite all there? Often one or two obvious ones are missing — not because they're irrelevant, but because your phrasing didn't land near them in vector space. **Write down a ref you expected but didn't see.** We'll watch it reappear in Lesson 1.4.

2. See the single-shot answer (needs a backend — OpenRouter key or local Ollama; skip if you have neither):

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם"
```

Read the answer. Notice its *shape*: does it weave the sources into one argument, or list them ("Source 1 says…, Source 2 says…")? Note that too — Lesson 2.1 changes it.

You should now be able to say…

- What "single shot" means precisely: **embed once → search once → generate once**, with no reconsidering, re-reading, or checking.

- Why that ceiling bites hardest on **conceptual** questions, where the answer's wording differs from the question's.
- The three shape-changes that lift it — **ask better, think before answering, check yourself** — and that none of them touch the refusal gate.

Next: **0.2 — The three moves (the map)** — a bird's-eye view of everything we added, before we open each box.

Lesson 0.2 — The three moves (the map)

> Module 0, Lesson 2 · ~10 min read + a config skim. The one question this answers: **what exactly did Prompt 31 add, and how do the pieces fit together into the upgraded `ask`?**

Before we zoom into each technique, hold the whole picture. Prompt 31 inserts three moves around the engine you already have. Here's the new shape, with the old one shown for contrast:

```
OLD:  question ──────────────► embed once ► search ► [gate] ► generate
──────────────► answer

NEW:  question ► EXPAND ► embed each ► search each ► FUSE ► [gate] ► ANALYZE
► SYNTHESIZE ► VERIFY ► answer
                  (1)                                          (2)
(2)        (3)
```

Three moves, numbered to the modules that teach them.

Move 1 — Expand (ask better) · Module 1

Instead of one query, produce several and fuse the results:

- **Lexicon expansion** — deterministic, no model call: if a curated term appears in the question, inject its canonical form and related terms.
- **Multi-query** — ask the model for a few rephrasings and sub-angles.
- **HyDE** — ask the model to draft a *hypothetical source passage*, and search with that.

All the resulting hit-lists are merged with **Reciprocal Rank Fusion** (RRF). A passage found by several queries rises; the net is wider, so good sources stop slipping through.

Move 2 — Reason (think before answering) · Module 2

Split the single generation into two stages:

- **Analyze** — read the numbered sources and produce a **study map**: each source's claim, then where they agree, build, and conflict.
- **Synthesize** — write one woven, cited answer *from* that map, instead of summarizing sources one by one.

Move 3 — Verify (check yourself) · Module 2.4

After the answer is written, optionally ask the model to flag any sentence its cited sources don't actually support. Flag-only — it never silently rewrites your answer.

The one thing none of them touch

The **default-deny gate** sits exactly where it always did — *after* retrieval, *before* any generation. Expansion feeds it more candidates but doesn't lower the bar; reasoning and verify happen only *after* the gate has decided to answer. If retrieval finds nothing relevant, the system still refuses **without calling the model at all** — same as Course 1. We prove this in Lesson 2.4.

That's the design rule of the whole upgrade: **add intelligence around the trust core, never through it.**

Everything is off by default

Each move is gated by config and defaults to **off**, so your system behaves exactly as it did after Course 1 until you opt in — per call with flags, or globally in `.env`.

Hands-on

Skim the new settings so the names are familiar when we use them. Open .env.example and find the two new blocks, or list them from config:

```
uv run python - <<'PY'
from maayan.config import Settings
s = Settings()
for k in ("query_expand_enabled", "query_expand_lexicon",
"query_expand_hyde",
          "query_expand_variants", "query_expand_max_queries",
          "rag_reasoning_enabled", "answer_verify_enabled"):
    print(f"{k:28} = {getattr(s, k)!r}")
PY
```

Note they're all `False`/defaults — the system is in its Course-1 behavior right now. Map each name to a move: the `query_expand_*` family is **Move 1**, `rag_reasoning_enabled` is **Move 2**, `answer_verify_enabled` is **Move 3**.

You should now be able to say…

- The new pipeline shape: **expand → fuse → [gate] → analyze → synthesize → verify**.
- Which move each config family controls, and that all default to **off**.
- The governing rule: intelligence is added **around** the default-deny trust core, never through it — retrieval still gates before any model call.

Next: **0.3** — **Feel the difference** — run the same question three ways and watch the upgrade happen.

Lesson 0.3 — Feel the difference

> Module 0, Lesson 3 · ~15 min, mostly hands-on (needs a generation backend). The one question this answers: **what does the upgrade actually feel like — same question, old way vs. new way?**

Concepts land harder once you've seen them. Before we open any box, run one question three ways and watch each move switch on. Keep this terminal session; we'll refer back to it.

> **Backend note.** The `--reason` runs make real model calls, so you need OpenRouter (`OPENROUTER_API_KEY`) or local Ollama configured. The `--expand` retrieval effect you can also see in `search`-style output with no backend, via lexicon-only expansion (Lesson 1.3).

The three runs

Pick the conceptual question you used in Lesson 0.1. Run these in order.

Run A — plain (Course 1 behavior):

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם"
```

This is single-shot: one query, one answer. Note the **Sources** list and the answer's shape.

Run B — add expansion:

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם" --expand
```

`--expand` turns on Move 1. Look at the **Sources** list now: it's the RRF-fusion of several queries (lexicon + reformulations + HyDE). Did a source appear that Run A missed? Did the order shift toward more obviously-relevant passages?

Run C — add reasoning, and reveal the study map:

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם" --expand --reason --
show-reasoning
```

`--reason` turns on Move 2; `--show-reasoning` prints the **study map** the model built before writing. Read the map first: one line per source, then the agreements/tensions. Then read the answer — notice it now *weaves* the sources (following the map) rather than listing them.

What you just saw

Run	Flags	What changed
A	*(none)*	The baseline: one query, one generation.
B	`--expand`	A wider, fused candidate set — better/more sources, often a different #1.
C	`--expand --reason --show-reasoning`	A two-stage answer: an explicit study map, then a woven, cited synthesis.

Add `--verify` to Run C and you'll also get a ⚠ **Claims not clearly supported…** section if the model finds any unsupported sentence — Move 3.

> **Under the hood — flags vs. `.env`**
>
> The `--expand/--reason/--verify` flags are *per-call overrides*. With no flag, each mode falls back to its config default (`QUERY_EXPAND_ENABLED` , etc.). So you can leave the system in Course-1 mode globally and opt into intelligence only when you want it — or flip the `.env` switches once you've decided (Module 4) that a mode earns its cost (Module 3).

Hands-on

1. Do the three runs above on **two** different questions: one conceptual (where you expect a big difference) and one whose answer is a single well-known passage (where you might not). Expansion and reasoning help most on the conceptual one — start noticing *when* the upgrade pays off. That instinct is what Module 3 makes quantitative.
2. Run C once more with `--no-reason` but keep `--expand` . Confirm the study map disappears but the fused sources remain — the two moves are independent.

You should now be able to say…

- What each flag does to a live `ask`, and that they compose independently.
- The difference you can *see*: expansion changes the **sources**; reasoning changes the **answer's shape** (and reveals a study map); verify adds a **flag list**.
- That flags are per-call overrides of the `.env` defaults — opt in without changing the system's resting behavior.

Next: **1.1 — Why a verbatim query under-retrieves** — open the first box.

Module 1 — Asking better (query intelligence)

Understand why one verbatim query is a weak net, and exactly how maayan widens it — deterministically with your lexicon, and generatively with reformulations + HyDE — then fuses the results.

Lesson 1.1 — Why a verbatim query under-retrieves

> Module 1, Lesson 1 · ~12 min read + a hands-on. The one question this answers: **why isn't embedding the question once and searching good enough — what does a single query systematically miss?**

Course 1, Lesson 1.1 taught you that an embedding places text in a space where *similar meaning = nearby*. That's true, and it's why retrieval works at all. This lesson is about its blind spot — the gap that query expansion (the rest of Module 1) exists to close.

One question is one point in space

When you ask, the retriever does this, once:

```
emb = self._embedder.embed_query(query)   # retrieve/retriever.py
```

Your whole question collapses to a **single** dense vector (plus a sparse one). Retrieval then returns the passages nearest *that one point*. Everything hangs on your exact phrasing landing near the passages that answer it.

For many questions it does. For conceptual ones, it often doesn't — and here's why.

The vocabulary-mismatch problem

The passage that answers a deep question frequently shares **no surface words** with the question, and may not even state the relationship you're asking about:

- You ask about a *relationship* ("מה הקשר בין…"). The source never says "relationship"; it just discusses idea A in one place and idea B in another, and a scholar knows they connect.
- You use a common word; the text uses an abbreviation, an Aramaic synonym, or a Kabbalistic term of art for the same concept.
- You ask in English; the answer is in Hebrew (or vice versa). `bge-m3` is multilingual, which helps — but cross-language matches still sit *farther apart* than same-language ones.

Each of these pushes the answer's vector **away** from your question's vector. A single search ranks by that one distance, so a genuinely-relevant passage can land at rank 12 — past your `top_k` — and simply never be seen. The generator can't cite what retrieval didn't return.

> **Under the hood — hybrid search helps, but doesn't solve this**
>
> You already fuse dense (meaning) and sparse (wording) search (Course 1, 2.3). Sparse catches exact shared terms, which mitigates *some* mismatch. But when the question and answer share *neither* meaning-vector proximity *nor* literal words — the conceptual case — both halves of hybrid search miss together. The fix isn't a better single query; it's **more than one query**.

The reframe: don't search smarter, search *wider*

If any *one* phrasing is a narrow net, cast several. Rephrase the question; inject the precise term; even draft a hypothetical answer and search with that. Each casts the net around a slightly different point in space, and together they cover the region where the real answer lives. Then fuse the catches.

That's the whole of Module 1:

- **1.2** — generative widening (multi-query + HyDE),
- **1.3** — deterministic widening with your lexicon,
- **1.4** — fusing the catches (RRF) and wiring it in transparently.

Hands-on

Reuse the conceptual question and the "expected but missing" ref you noted in Lesson 0.1.

1. Confirm the miss, and find its cause. Run the question and also a query using the *words you'd expect the source itself to use*:

```
uv run maayan search "מה הקשר בין צמצום לבריאת העולם" --k 10
uv run maayan search "צמצום אור אין סוף מקום פנוי" --k 10
```

The second query — phrased like the *source*, not like a *question* — likely surfaces passages the first missed, higher up. That delta is the vocabulary-mismatch problem in your own corpus: same information need, different words, different results.

2. Note the asymmetry. Pick a passage you know well and search once with a paraphrase and once with a near-quote of it. The near-quote ranks it higher. Real users ask in paraphrase — which is exactly why we need to manufacture the near-quote phrasings ourselves (next two lessons).

You should now be able to say…

- Why a single query is a **single point** in vector space, and retrieval ranks by distance to it.
- The **vocabulary-mismatch** problem: conceptual questions share neither proximity nor words with their answers, so relevant passages fall past `top_k`.
- Why hybrid search mitigates but doesn't solve it, and why the fix is **searching wider** (many queries), not searching smarter (one better query).

Next: **1.2 — Multi-query & HyDE** — the generative way to widen the net.

Lesson 1.2 — Multi-query & HyDE

> Module 1, Lesson 2 · ~15 min read + a hands-on (needs a backend for the live runs). The one question this answers: **how do we use the language model to turn one question into several good search queries — and why does searching with a *fake answer* work so well?**

Lesson 1.1 left us wanting several phrasings instead of one. The model that writes your answers is also excellent at *rephrasing* and *imagining* — so we put it to work before retrieval, too. Two techniques, both in `LLMQueryExpander` (retrieve/expand.py).

Multi-query: rephrase the question several ways

The first technique is the simplest: ask the model for alternative search queries.

```
MULTI_QUERY_SYSTEM_PROMPT = (
    "You help search a corpus of chassidus and Kabbalah ... write
alternative search "
    "queries ... vary the wording and angle (synonyms, the underlying
concept, related "
    "technical terms, Hebrew/English). ... output ONE query per line ...
do not answer."
)
```

The expander sends your question, gets back a handful of lines, strips any bullets/numbering (`_clean_lines`), and keeps up to `query_expand_variants` of them. Each line lands at a slightly different point in vector space — a synonym here, the underlying concept there, the English rendering — so together they cover more of the region where the answer lives.

The instruction "**do not answer the question**" matters: we want *queries*, not prose. (The next technique deliberately does the opposite.)

HyDE: search with a hypothetical answer

HyDE — **Hy**pothetical **D**ocument **E**mbeddings — is the clever one. Instead of searching with the *question*, draft a short passage that would *answer* it, and search with **that**:

```
HYDE_SYSTEM_PROMPT = (
    "... write a short, plausible passage (2-4 sentences) of the kind a
source text might "
    "contain that would answer it — in the language of the question. This
is a search aid, "
    "not an answer: do NOT add citations, and it is fine if some details
are uncertain."
)
```

Why a *fake* answer beats the real question: retrieval matches **answer-shaped text against answer-shaped text**. Your question ("what is the relationship between…?") is shaped nothing like a source passage. A hypothetical answer is shaped *exactly* like one — same register, same vocabulary, same concepts asserted rather than asked — so its embedding lands right in the neighborhood of the genuine sources. It doesn't matter if the hypothetical is partly wrong; we're not citing it, only using its *position in space* to find the real thing. The real passages then get retrieved and become the only citable sources (default-deny still rules).

> **Under the hood — two calls, by design**
>
> `LLMQueryExpander.expand` makes up to **two** model calls: one for the reformulations, one for the HyDE passage. Each call has a single, clean job, which keeps parsing trivial and the prompts focused. That's part of the cost ladder we count in Lesson 3.1 — and why HyDE has its own toggle (`query_expand_hyde`) you can switch off when you want a cheaper expand.

Hands-on

These call the model, so set a backend first (OpenRouter key or Ollama). See the variants the model actually generates for one of your questions:

```
uv run python - <<'PY'
from maayan.config import Settings
from maayan.generate.factory import build_generation_backend
from maayan.retrieve.expand import LLMQueryExpander

backend = build_generation_backend(Settings())
ex = LLMQueryExpander(backend, variants=3, hyde=True)
out = ex.expand("מה הקשר בין צמצום לבריאת העולם")
for i, q in enumerate(out.queries):
    tag = "original" if i == 0 else ("HyDE" if i == len(out.queries) - 1
else "variant")
    print(f"[{tag:8}] {q}")
PY
```

Read them critically:

- Are the **variants** genuinely different angles, or near-duplicates? (If duplicative, your model is weak at this — lower `variants`, or lean on lexicon expansion in 1.3.)
- Is the **HyDE** passage source-shaped — does it *read like Tanya*, asserting rather than asking? That's the signal it'll retrieve well.

Then feel the payoff: search with the HyDE passage you just printed and compare to searching with the bare question. The HyDE search should rank the real sources higher.

You should now be able to say…

- **Multi-query**: ask the model for several rephrasings/angles; each is a different point in space; we keep up to `query_expand_variants`.
- **HyDE**: search with a hypothetical *answer*, because answer-shaped text matches the sources better than question-shaped text — and being partly wrong is fine, since we never cite it.
- Why the expander makes two clean model calls, and that HyDE is independently toggleable.

Next: **1.3 — Lexicon-aware expansion** — the free, deterministic widening that's uniquely yours.

Lesson 1.3 — Lexicon-aware expansion (no model needed)

> Module 1, Lesson 3 · ~12 min read + a hands-on (no backend required). The one question this answers: **how does maayan use the lexicon *you* curated to widen a query for free — and why is this the most "yours" part of the whole upgrade?**

The generative expanders (1.2) are powerful but cost model calls and depend on the model's quality. There's a second, complementary expander that costs **nothing** and gets *better the more you teach the system*: `LexiconExpander` (retrieve/expand.py).

The idea: inject curated vocabulary on a match

Recall the lexicon from Course 1 (Module 6): you defined **terms** — Holy Names, sefiros, technical concepts — each with a canonical form, surface variants, related terms, and a definition, stored in `TermStore` and indexed as `source="term"` chunks. The lexicon expander puts that knowledge to a new use:

> When a registered term appears in the question, append one extra query that augments the question with that term's **canonical form** and **related terms**.

So if you've defined `ע"ב` (Name of 72) with related terms `ס"ג / מ"ה / ב"ן`, then asking about `ע"ב` automatically also searches for the whole family — pulling in passages that discuss the siblings, which a scholar would absolutely want, but which your raw query never named.

How the match works (and why it's safe)

It reuses the exact tolerant matching the lexicon already uses — `fold_surface` from `corpus/normalize.py` (Course 1, 1.3): drop nikkud, strip gershayim/quotes, casefold. So , `ע"ב` `ע״ב`, and `עב` all match the same term, regardless of how you typed the question.

```
folded_query = fold_surface(query)
for term in self._terms.list_terms():
    if term.retracted:
        continue                    # never resurface a retracted term
    forms = term.surface_forms or [term.canonical]
    if any(fold_surface(f) in folded_query for f in forms):
        additions.append(term.canonical)
        additions.extend(term.related_terms)
```

Three things worth noticing, each deliberate:

- **Deterministic.** Same query + same lexicon → same expansion, every time. No model, no randomness, no latency. It's the cheapest possible way to widen the net.
- **Retracted terms are skipped.** If you tombstoned a term (Course 1, Phase 4), it won't sneak back in through expansion. The eraser stays erased.
- **It compounds with your work.** Every term you add to the lexicon makes *every future query* that mentions it a little smarter. The capture loop now improves retrieval, not just the answer corpus — a quiet but real payoff.

> **Under the hood — a clean DI seam**
>
> `LexiconExpander` doesn't depend on the concrete `TermStore`; it takes anything satisfying the tiny `TermSource` protocol (`list_terms() -> list[Term]`). That's why the test can hand it a fake list and why you could back it with a different store later — the same dependency- injection discipline from Course 1, Module 5.

Hands-on

No backend needed — this is pure Python over your SQLite lexicon.

1. Expand a query that mentions a term you've defined. Substitute a real term/surface form from your lexicon:

```
uv run python - <<'PY'
from maayan.config import Settings
from maayan.lexicon.store import TermStore
from maayan.retrieve.expand import LexiconExpander

s = Settings()
ex = LexiconExpander(TermStore(s.db_path))
out = ex.expand('מהו ע"ב')          # <- use a surface form you actually
defined
for q in out.queries:
    print(repr(q))
PY
```

If the term matched, you'll see a second query carrying its canonical form and related terms. If nothing expanded, either the term isn't in your lexicon or its surface forms don't fold to match — a good prompt to go define/improve it (and a reminder of why curation pays off here).

2. Prove the payoff loop. Define a new term with a couple of related terms (`maayan term add …`, Course 1), then re-run step 1 with a query naming it. The expansion grows the moment you teach it. *That* is the capture loop feeding retrieval.

You should now be able to say…

- What `LexiconExpander` does: on a folded-surface match, inject the term's canonical + related terms as an extra query — deterministically, with no model call.
- Why it's safe (skips retracted terms) and tolerant (reuses `fold_surface`).
- Why it's the most *yours* expander: every lexicon entry you curate makes future retrieval smarter — the capture loop now improves search, not just the answer corpus.

Next: **1.4 — Fusing the nets: RRF + the drop-in retriever** — how all these queries become one ranked list without changing anything downstream.

Lesson 1.4 — Fusing the nets: RRF + the drop-in retriever

> Module 1, Lesson 4 · ~18 min read + a hands-on (no backend required). The one question this answers: **once we have several queries, how do their results become one ranked list — and how does this slot into the system without touching `ask`, `develop`, or threads?**

We can now widen the net three ways (1.2–1.3). Each query returns its own ranked list. This lesson merges them and — just as importantly — shows how the whole thing plugs in *invisibly*.

RRF, again — now across queries, not vector types

You already met Reciprocal Rank Fusion in Course 1 (2.3), where it fused **dense vs. sparse** results for one query. Here it does the same job one level up: fuse the result lists of **several queries**. The math is identical and lives in retrieve/fuse.py:

```
fused[ref] += 1.0 / (rrf_k + rank)     # sum over every list the ref
appears in
```

Each result contributes `1/(rrf_k + rank)` from each list it appears in (rank is 0-based, `rrf_k=60`). A passage retrieved by **several** of your queries accumulates from each and rises; one found by a single query still gets credit, but less. So fusion rewards exactly what you want: sources that *multiple* phrasings agree are relevant.

It's pure and deterministic — deduped by `ref`, ties broken by `ref` ascending (same reproducibility discipline as Course 1). No model, fully unit-tested in `tests/test_fuse.py`.

> **Under the hood — RRF rewards agreement, sometimes surprisingly**
>
> Because `rrf_k=60` is large, RRF is nearly linear: two appearances at rank 1 (`2/61`) can outscore one appearance at rank 0 (`1/60`). That's intentional — *consensus across queries* is a strong signal — but it means a chunk every variant retrieves in its mid-list can rise above a chunk one variant loved. If you ever want top-rank to dominate more, that's the `rrf_k` dial (smaller = top ranks matter more).

Keeping the refusal gate honest: `relevance = max`

Here's the subtle part. The fused `score` is rank-based — RRF *always* ranks something first, even for a nonsense question (Course 1, 2.3 taught you this). So the fused score **cannot** be the number the default-deny gate checks. The `MultiQueryRetriever` handles this exactly right:

```
    for q in queries:
        sub = self._base.retrieve(q, ...)
        result_lists.append(sub.results)
        relevance = max(relevance, sub.relevance)      # absolute, per-
    variant
    fused = rrf_fuse(result_lists, limit=k or self._top_k)
    return RetrievalResult(results=fused, relevance=relevance)
```

It fuses the *ranked lists* for ordering, but carries **`relevance = the maximum absolute relevance across the variants`** — i.e. "did *any* phrasing of the question find something genuinely relevant?" That's the honest question for the gate, and it's still the absolute dense- cosine measure from Course 1, never the RRF score. Expansion widens the net; it does **not** lower the bar for answering.

The drop-in: nothing downstream changes

`MultiQueryRetriever` implements the **same `Retrieving` protocol** as the base `Retriever` (Course 1, Module 5 — program to the interface). It wraps a base retriever + an expander:

```
MultiQueryRetriever.retrieve(q)
   └─ expander.expand(q) → [q, variant1, …, HyDE, lexicon-aug]
   └─ for each: base.retrieve(...)         # the ordinary hybrid+rerank
pipeline
   └─ rrf_fuse(lists)  +  relevance = max
```

Because it *is* a `Retrieving`, every caller — `RAGService.ask`, `DevelopmentService`, the thread flow — uses it with **zero changes**. The factory decides which to build:

```
    # retrieve/factory.py
    if not use_expand:
        return base                                # plain Retriever, exactly
    as Course 1
    ...
    return MultiQueryRetriever(base, CompositeExpander(expanders, ...),
    top_k=resolved_top_k)
```

This is the DI payoff from Course 1 made concrete: a substantial new capability added behind an existing seam, invisible to everything that depends on it.

Hands-on

No backend needed — use lexicon-only expansion so it runs offline.

1. Watch a fused source appear. Compare the base retriever to the expanded one on your Lesson 0.1 question (use one that mentions a lexicon term, so lexicon expansion fires):

```
uv run python - <<'PY'
from maayan.config import Settings
from maayan.retrieve.factory import build_retriever

s = Settings()
q = 'מהו ע"ב'                       # a query naming a term you've
defined
base = build_retriever(s, expand=False)
expanded = build_retriever(s, expand=True)        # backend=None →
lexicon-only

base_refs = [r.ref for r in base.retrieve(q, k=8).results]
exp_refs  = [r.ref for r in expanded.retrieve(q, k=8).results]
print("base    :", base_refs)
print("expanded:", exp_refs)
print("NEW in expanded:", [r for r in exp_refs if r not in base_refs])
print("base relevance vs expanded:",
      round(base.retrieve(q, k=8).relevance, 3),
      round(expanded.retrieve(q, k=8).relevance, 3))
PY
```

The "NEW in expanded" line is the source(s) the related-term family pulled in — quite possibly the ref you marked missing in Lesson 0.1. Note the `relevance` numbers: expanded's is `max` across variants, so it's `>=` base's — never lower.

2. Confirm the drop-in. Notice you never touched `ask` to get this — `build_retriever(..., expand=True)` returned a different object behind the same `Retrieving` interface. That's the whole trick.

You should now be able to say…

- How RRF fuses several queries' results into one ranking, and why it rewards cross-query agreement (and the `rrf_k` caveat).

- Why the gate uses **`relevance = max` across variants** (absolute), never the rank-based fused score — expansion widens the net without lowering the bar.
- How `MultiQueryRetriever` drops in behind the `Retrieving` protocol so `ask` / `develop` /threads are unchanged — DI from Course 1, paying off.

Next: **2.1 — One pass vs. two stages** — we move from finding the sources to *thinking* about them.

Module 2 — Thinking before answering (reasoning & synthesis)

Understand the two-stage answer — analyze the sources into a study map, then weave a grounded answer from it — why it suits chassidus, and how the trust core stays intact.

Lesson 2.1 — One pass vs. two stages

> Module 2, Lesson 1 · ~12 min read + a hands-on (needs a backend). The one question this answers: **what changes when the model *thinks before it writes* — and why does splitting one generation into two raise the quality of the answer?**

Module 1 got the right sources in front of the model. Module 2 is about what the model *does* with them. The single-shot system did one thing: sources in, answer out. The reasoning path does two — and the split is the point.

The shape change

Open `RAGService.ask` in generate/rag.py. With reasoning on, the body branches:

```
if self._reasoning:
    reasoning_text = self._backend.generate(self._analyze_prompt,
build_analyze_messages(sources, question))
    user_content   = self._synthesis_user_content(question, sources,
reasoning_text, context_turns)
else:
    user_content   = self._answer_user_content(question, sources,
context_turns)
text = self._backend.generate(self._system_prompt, [Message(role="user",
content=user_content)])
```

- **Stage 1 — ANALYZE.** A first model call reads the numbered sources and produces a *study map* (Lesson 2.2): each source's claim, then how they relate.
- **Stage 2 — SYNTHESIZE.** A second call writes the answer, but now it's handed the sources **and** the study map, with instructions to *weave* (Lesson 2.3).

Single-shot is still the default (the `else` branch is byte-for-byte the Course 1 path). Turning on `--reason` just inserts Stage 1 and feeds its output into Stage 2.

Why two passes beat one

It's the same reason a person outlines before writing, or a chavrusa lays out the sugya before drawing a conclusion. Asking a model to *understand* and *compose* in a single step makes it do both at once,

and it tends to shortchange the first — producing a tour of the sources ("Source 1 says X; Source 2 says Y") instead of an argument that uses them.

Separating the steps:

- **Forces structure first.** Stage 1's only job is to map the terrain — claims, agreements, tensions — with nothing else competing for attention. A better map means a better answer.
- **Gives Stage 2 a scaffold.** The synthesizer isn't staring at raw quotes; it's working from an explicit relational map, so it can *connect* rather than *list*.
- **Makes the reasoning inspectable.** The study map is returned to you (`Answer.reasoning`), so you can see *why* the answer says what it does — and catch a bad answer at its root (a wrong map) instead of guessing.

The cost is one extra model call per ask (Lesson 3.1 counts the whole ladder). That's the trade: more tokens, a better and more transparent answer. You decide per question, per corpus.

Hands-on

Backend required. Use a conceptual question with several relevant sources.

1. Compare the shapes directly.

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם"
# one pass
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם" --reason            #
two stages
```

Read both answers side by side. The single-pass one often *enumerates* sources; the reasoned one more often *integrates* them ("X grounds Y, though Z qualifies it"). Note which you'd rather hand a student.

2. Confirm it's two calls. A quick offline proof that reasoning adds exactly one generation, using a fake retriever + a backend that counts calls:

```
uv run python - <<'PY'
from maayan.generate.rag import RAGService
from maayan.retrieve.models import RetrievalResult, SearchResult

class FakeRetriever:
    def retrieve(self, q, *, k=None, book=None, source=None):
        r = SearchResult(ref="Tanya 1:1", text="…", score=0.5, lang="he",
source="sefaria", payload={"ref":"Tanya 1:1"})
        return RetrievalResult(results=[r], relevance=0.9)

class Counter:
    def __init__(self): self.n = 0
    def generate(self, system, messages): self.n += 1; return "answer
[S1]."

for reasoning in (False, True):
    b = Counter()
    RAGService(FakeRetriever(), b, score_threshold=0.4,
reasoning=reasoning).ask("q")
    print(f"reasoning={reasoning}: {b.n} model call(s)")
PY
```

You'll see `1` then `2`. That single extra call is the whole price of Stage 1.

You should now be able to say…

- The two-stage shape: **ANALYZE** (sources → study map) then **SYNTHESIZE** (sources + map → answer), versus the single-shot default.
- *Why* two passes beat one: structure-first produces integration instead of enumeration, and the map makes the reasoning inspectable.
- The cost: exactly one extra model call, paid only when you opt in.

Next: **2.2** — **The study map** — what Stage 1 actually produces, and why it's the most chassidus-shaped move in the system.

Lesson 2.2 — The study map

> Module 2, Lesson 2 · ~12 min read + a hands-on (needs a backend). The one question this answers: **what is the "study map" Stage 1 produces, and why is it the most chassidus-shaped part of the entire system?**

Stage 1 of the reasoning path has exactly one job: read the retrieved sources and lay out how they relate. The artifact it produces — the **study map** — is small, but it's where "lookup" becomes "learning."

What ANALYZE is told to do

The prompt is deliberately narrow (generate/rag.py):

```
ANALYZE_SYSTEM_PROMPT = (
    "... Read the numbered SOURCES and produce a concise STUDY MAP ...\n"
    "1. For each source, one line: its tag then its core claim in your
words, e.g. '[S1] — ...'.
        Use ONLY what the source says; add no outside facts.\n"
    "2. Then, briefly: where sources AGREE, where one BUILDS ON another,
and any TENSION/
        disagreement between them. Refer to sources by their [S#]
tags.\n"
    "3. Do NOT answer the question yet and do not invent mekoros. Keep it
tight."
)
```

Two parts: a one-line **claim per source**, then the **relationships** — agree / builds-on / tension. The user message (`build_analyze_messages`) hands it the numbered sources *and* the question, so the map is oriented toward what's being asked, not a generic summary.

Three guardrails are baked in, and they're the same trust rules as everywhere else: *only what the source says*, *refer by [S#]*, *don't invent mekoros*, *don't answer yet*. The map is analysis, not opinion.

Why this is the chassidus move

Stop and notice what the second part of the map asks for: **where sources agree, where one builds on another, and where they're in tension.** That is precisely the work of a chavrusa learning a sugya

— not "what does each source say" (a student can read), but "**how do they fit together**." A maamar develops by building one source on another, resolving an apparent tension, showing that two statements are really one idea at different levels. The study map makes the system attempt *that* move explicitly, before it commits to an answer.

This is why reasoning matters more here than in a generic Q&A bot. For factual lookup, "how the sources relate" is often trivial. For p'nimiyus haTorah, it's the whole game. The study map is the system's first genuine attempt to *learn*, not just *retrieve* — and because it's returned to you, you can judge that attempt directly.

> **Under the hood — the map is never citable, and never the answer**
>
> The study map is the model's *own* analysis, so the next stage is forbidden to cite it (Lesson 2.3): only the numbered `[S#]` sources are citable. And ANALYZE is told "do NOT answer the question yet" — if you ever see the map starting to *argue* rather than *map*, that's a prompt you can tighten (Lesson 4.2). Keeping the two stages cleanly separated is what makes each one good.

Hands-on

Backend required.

1. Read a study map. Print it with `--show-reasoning`:

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם" --reason --show-
reasoning
```

The **Study map:** block prints before the answer. Read it as a scholar would:

- Is each `[S#]` claim *faithful* to that source (cross-check one against the **Sources** list)?
- Do the agreement/tension notes ring true, or is the model forcing a connection? (Forced connections are the failure mode to watch — Lesson 4.2.)

2. See the map drive the answer. Re-read the answer and find where it *follows* the map — e.g., the map says "[S2] builds on [S1]," and the answer presents them in that order, connected. When the answer is good, you can usually trace it back to a good map; when it's off, the map usually shows you why. That traceability is the practical payoff of making reasoning explicit.

You should now be able to say…

- What the study map contains: one faithful claim per source, then agreements / builds-on / tensions, oriented to the question.
- Why it's the most chassidus-shaped move: "how the sources fit" is the substance of learning p'nimiyus haTorah, and the map attempts exactly that.
- That the map is analysis (never citable, never the answer) and is returned to you so the reasoning is inspectable.

Next: **2.3 — Synthesis: weave, don't list** — turning the map into a grounded answer.

Lesson 2.3 — Synthesis: weave, don't list

> Module 2, Lesson 3 · ~12 min read + a hands-on (needs a backend). The one question this answers: **how does Stage 2 turn the study map and the sources into a single woven answer — while keeping every claim grounded and cited?**

Stage 1 built the map (2.2). Stage 2 writes the answer from it. The whole design goal of this stage is one word: **weave** — connect the sources into one argument, rather than narrate them one at a time.

What the synthesizer receives

The system prompt for Stage 2 is the *same* grounded-answer prompt from Course 1 (`DEFAULT_SYSTEM_PROMPT`: answer only from the numbered sources, cite every claim with `[S#]`, refuse if unsupported). What's new is the **user message**, assembled by `_synthesis_user_content` (generate/rag.py):

```
blocks = [ (conversation, if any),
           build_context(sources),                      # the numbered
[S#] sources
           "STUDY MAP (your own analysis ... use it to organize and
connect, "
           "but cite ONLY the [S#] sources, never the map):\n" +
study_map,
           f"Question: {question}\n\n"
           "Using the study map, weave the sources into a single coherent
answer "
           "(connect them — do not just list them). Cite each claim ONLY
by its [S#] tag. "
           "Do not cite the conversation or the study map." ]
```

So the synthesizer sees three things: the **sources** (citable), the **study map** (its scaffold, *not* citable), and an instruction to **weave and cite**. The map tells it *how the pieces connect*; the sources are what it's allowed to stand on.

The two disciplines that keep it honest

This stage adds connective intelligence **without loosening grounding**. Two rules do that:

1. **Cite only `[S#]` sources — never the map.** The study map is the model's own words; letting it be cited would let the model cite *itself*. The prompt forbids it explicitly, and citation extraction (`extract_cited_refs`, unchanged from Course 1) only resolves `[S#]` tags back to real source refs. So "weave" never becomes "embellish."
2. **Same refusal rule inside the prompt.** The synthesis system prompt still says: if the sources don't support it, say so. Reasoning makes the answer *better-organized*, not *bolder*. (And the hard, code-level default-deny gate already fired before we ever got here — Lesson 2.4.)

The result: the answer reads like a connected explanation — "X establishes the principle [S1], which Y extends to creation [S3], resolving the difficulty Z raised [S2]" — while every clause still traces to a real mekor.

> **Under the hood — why reuse the Course 1 system prompt?**
>
> Stage 2 deliberately keeps `DEFAULT_SYSTEM_PROMPT` rather than inventing a new one. The grounding/citation/refusal *rules* should be identical whether or not reasoning is on — only the *material* differs (now there's a study map to weave from). Reusing the prompt guarantees the trust contract doesn't drift between modes. It also means the single-pass path and the synthesis path cite identically, so `cited_refs` means the same thing everywhere.

Hands-on

Backend required.

1. Inspect the exact synthesis prompt. See precisely what Stage 2 is handed (sources + map + instruction), using a backend that records its calls:

```
uv run python - <<'PY'
from maayan.generate.rag import RAGService
from maayan.retrieve.models import RetrievalResult, SearchResult

class FakeRetriever:
    def retrieve(self, q, *, k=None, book=None, source=None):
        rs = [SearchResult(ref=f"Tanya 1:{i}", text=f"source text {i}",
score=0.5,
                           lang="he", source="sefaria", payload={"ref":
f"Tanya 1:{i}"}) for i in (1,2)]
        return RetrievalResult(results=rs, relevance=0.9)

class Recorder:
    def __init__(self): self.calls=[]
    def generate(self, system, messages):
        self.calls.append(messages[0].content)
        return "STUDY MAP: [S1] …; [S2] builds on [S1]." if "STUDY MAP"
in system else "Woven answer [S1][S2]."

b = Recorder()
RAGService(FakeRetriever(), b, score_threshold=0.4,
reasoning=True).ask("the question")
print(b.calls[1])      # the SYNTHESIS user message
PY
```

Find the three blocks in the printed message: `SOURCES:` (the `[S#]` list), `STUDY MAP (...)` (the scaffold), and the `Question: … weave …` instruction. That's the entire contract Stage 2 works under.

2. Judge a real weave. Run a live `--reason` answer and check: does every factual clause carry a `[S#]`? Does it *connect* sources (using words like "therefore," "extends," "however") rather than list them? Those two together — grounded *and* woven — are the bar this stage aims for.

You should now be able to say…

- What Stage 2 receives: sources (citable) + study map (scaffold, not citable) + a weave-and- cite instruction.
- The two disciplines that keep "weave" honest: cite only `[S#]`, and keep Course 1's refusal rule — reasoning improves organization, not boldness.
- Why Stage 2 reuses the Course 1 system prompt: so the grounding contract never drifts between single-pass and reasoning modes.

Next: **2.4 — Checking yourself, and the rule that didn't change**.

Lesson 2.4 — Checking yourself, and the rule that didn't change

> Module 2, Lesson 4 · ~12 min read + a hands-on (no backend required for the key proof). The one question this answers: **how does the optional verify pass catch unsupported claims — and how do we *know* that all this new intelligence never weakened the refusal guarantee?**

Two ideas close Module 2: a new safety net (verification), and the old safety net we must prove is still intact (default-deny). The second matters more than the first.

Verification: flag what the sources don't support

Reasoning and synthesis make the answer better-organized — but the model can still overstate. The optional **verify** pass (Move 3) is a final, separate check:

```
VERIFY_SYSTEM_PROMPT = (
    "You are a citation checker. You are given numbered SOURCES and an
ANSWER that cites them "
    "with [S#] tags. List any sentence ... NOT supported by the source(s)
it cites ... one per "
    "line ... If every claim is properly supported, output exactly: OK"
)
```

After the answer is written, `ask` makes one more model call (`build_verify_messages` hands it the sources + the answer), and `parse_unsupported` turns the reply into a list:

```
def parse_unsupported(reply: str) -> list[str]:
    stripped = reply.strip()
    if not stripped or stripped.upper() == "OK":
        return []
    return [line.strip() for line in stripped.splitlines() if
line.strip()]
```

Those land on `Answer.unsupported_claims`, and the CLI prints them under a ⚠ **Claims not clearly supported…** heading. Crucially, it's **flag-only** — the system never silently rewrites your answer. It surfaces doubt for *you* to judge; it doesn't act on your behalf. That fits the whole project's stance: the human stays in the loop.

This is a different kind of grounding than the prompt-level "cite your sources." The prompt *asks* the model to be faithful; verify *checks* whether it was — with fresh eyes, in a separate call that only sees the sources and the finished answer. Belt, meet suspenders.

The rule that did not change: default-deny

Now the important part. We added expansion, two-stage reasoning, and verification — up to five model calls (Lesson 3.1). Through all of it, **the refusal gate sits exactly where it always did**, and still fires *before any model call*. Look at the top of `ask`:

```
retrieval = self._retriever.retrieve(question, ...)
sources = retrieval.results
if not sources or retrieval.relevance < self._score_threshold:
    return Answer(..., grounded=False, text=self._refusal_text)   # NO
model call
# only past here do analyze / synthesize / verify ever run
```

Read the order carefully: retrieve → check `relevance` against the threshold → **return the refusal without generating** if it doesn't clear. Expansion feeds this gate a (possibly wider) candidate set and a `relevance = max` across variants — but it cannot *lower* the threshold. Reasoning and verify live entirely *after* the gate. So the guarantee from Course 1 holds verbatim: **if retrieval finds nothing relevant, the model is never called, in any mode.**

This was a design rule, not an accident: *add intelligence around the trust core, never through it.* It's why the new behavior could ship default-off with zero risk to the system's defining promise.

Hands-on

1. Prove default-deny holds with everything on — no backend needed. Force empty retrieval and confirm the model is never touched, even with reasoning *and* verify enabled:

```
uv run python - <<'PY'
from maayan.generate.rag import RAGService
from maayan.retrieve.models import RetrievalResult

class EmptyRetriever:
    def retrieve(self, q, *, k=None, book=None, source=None):
        return RetrievalResult(results=[], relevance=0.0)

class Boom:
    def generate(self, system, messages):
        raise AssertionError("model was called — default-deny FAILED")

ans = RAGService(EmptyRetriever(), Boom(), score_threshold=0.4,
                 reasoning=True, verify=True).ask("a question with no
sources")
print("grounded:", ans.grounded)          # False
print("text    :", ans.text)              # the refusal
print("no exception raised → model was NEVER called ✓")
PY
```

The `Boom` backend raises if touched; it isn't. That's the guarantee, demonstrated in code — the same proof the test suite makes (`test_reasoning_default_deny_makes_no_calls`).

2. See verify flag something (backend required). Run a `--reason --verify` answer on a hard question; if the model overstates anywhere, you'll see the ⚠ list. On a well-supported answer, you'll see nothing — `OK` parses to an empty list.

You should now be able to say…

- What verify does: a separate, final model call that flags unsupported sentences (`OK` → none), flag-only — never rewriting your answer.
- Why default-deny is **unchanged**: the gate runs before any model call, expansion can't lower the threshold, reasoning/verify run only after — so "no source → no model call" still holds in every mode.
- The governing principle of the whole upgrade: **intelligence around the trust core, never through it.**

Next: **3.1 — The cost ladder** — what all this intelligence actually costs, and how to decide when to spend it.

Module 3 — Proving the lift (evaluation & cost)

Replace “this feels smarter” with numbers and a clear sense of what each mode costs.

Lesson 3.1 — The cost ladder

> Module 3, Lesson 1 · ~10 min read + a hands-on (no backend required). The one question this answers: **what does each mode actually cost in model calls, and how do I decide when the intelligence is worth it?**

Every new move buys quality with model calls (tokens, latency, and — on a cloud backend — money). Knowing the exact price is what lets you choose modes with intent instead of leaving everything maxed "to be safe." Here's the whole ladder.

Counting the calls

A single `ask` makes generation calls in up to three places:

Stage	When	Calls
Expansion (`LLMQueryExpander`)	`--expand` with a backend	up to **2** — one for reformulations (`variants>0`), one for HyDE (`hyde=true`)
Answer	always (single-pass) **or**	**1** synthesize
→ split into **analyze + synthesize**	`--reason`	**2** instead of 1
Verify	`--verify`	**1**

So the ladder, cheapest to dearest:

```
plain ............................ 1 call
plain + verify ................... 2 calls
reason ........................... 2 calls   (analyze + synthesize)
reason + verify .................. 3 calls
expand + reason + verify ......... up to 5 calls  (2 expand + 2 reason + 1
verify)
```

Two things to note. **Lexicon expansion is free** — it's deterministic, zero model calls — so `--expand` with `variants=0, hyde=false` adds *nothing* to the bill while still widening the net with your terms (the "lexicon-only" recipe in Lesson 4.1). And **expansion calls are cheap calls**: short prompts, short outputs. The expensive call is usually synthesis over many sources.

Choosing a mode

A rule of thumb, by situation:

- **Browsing / quick lookups, or a factual question with one obvious source** → plain (or expand-lexicon-only). Reasoning adds little when there's nothing to *relate*.
- **A real conceptual question you'll act on** → `--expand --reason`. This is the sweet spot: better sources, a woven answer, ~4 calls.
- **Something you'll quote to others / publish** → add `--verify`. Pay the extra call to catch an overstatement before a person relies on it.
- **A weak/local model** → keep `variants` low and consider skipping reason (small models produce thin study maps — Lesson 4.1).

The point of off-by-default is precisely this: you spend calls where they earn their keep, and the system's resting cost is exactly Course 1's.

Hands-on

No backend needed — count calls with a counting fake across the ladder:

```
uv run python - <<'PY'
from maayan.generate.rag import RAGService
from maayan.retrieve.models import RetrievalResult, SearchResult

class FakeRetriever:
    def retrieve(self, q, *, k=None, book=None, source=None):
        r = SearchResult(ref="Tanya 1:1", text="…", score=0.5, lang="he",
source="sefaria", payload={"ref":"Tanya 1:1"})
        return RetrievalResult(results=[r], relevance=0.9)

class Counter:
    def __init__(self): self.n=0
    def generate(self, system, messages): self.n+=1; return "OK" if
"citation checker" in system else "answer [S1]."

for label, kw in [("plain", {}), ("verify", {"verify":True}),
                  ("reason", {"reasoning":True}), ("reason+verify",
{"reasoning":True,"verify":True})]:
    b = Counter()
    RAGService(FakeRetriever(), b, score_threshold=0.4, **kw).ask("q")
    print(f"{label:14} → {b.n} call(s)")
PY
```

(This counts the *answer-side* ladder; expansion adds up to 2 more at retrieval time, which you saw generated in Lesson 1.2.) Match the output to the table above.

You should now be able to say…

- The model-call cost of each mode, end to end (1 → up to 5), and that **lexicon expansion is free**.
- That expansion calls are cheap and synthesis is the costly one.
- A situational rule for choosing a mode — and why off-by-default lets you spend calls only where they pay.

Next: **3.2 — Measuring the lift with `eval-expand`** — proving expansion helps with numbers, not vibes.

Lesson 3.2 — Measuring the lift with `eval-expand`

> Module 3, Lesson 2 · ~12 min read + a hands-on. The one question this answers: **how do I prove query expansion actually retrieves better — with numbers — and learn when it helps and when it doesn't?**

Course 1, Module 4 taught you to replace "it feels right" with hit@k / recall@k / MRR over a gold set. Prompt 31 adds a command that points that same machinery at one question: **does expansion retrieve better than no expansion?** Never enable a mode on vibes — measure it.

What `eval-expand` does

`make eval-expand` (the `eval-expand` CLI command) runs your gold set through the retriever **twice** — once with expansion off, once on — and prints the two side by side:

```
uv run maayan eval-expand
# or, where expansion should help most:
uv run maayan eval-expand --crosstext
```

Under the hood it reuses Course 1's `run_eval` harness (eval/harness.py), because `MultiQueryRetriever` *is* a `Retrieving` (Lesson 1.4) and plugs straight in. It builds two retrievers via the factory — `expand=False` and `expand=True` — shares one embedder, and labels the rows `no-expand` vs `expand`. If a generation backend is configured it uses the full lexicon+LLM expansion; if not, it falls back to **lexicon-only** and tells you so (so you can run it offline).

You read the same columns as Course 1: `hit@k`, `recall@k`, `MRR`, and the default-deny gate rates (`answ` / `refus`).

Reading the result — and when expansion helps

The number to watch is **recall@k**: of the passages a question *should* retrieve, how many did it? Expansion's whole job is to stop relevant sources slipping past `top_k`, so recall is where the lift shows up.

- **Conceptual & cross-text questions** (answers spread across books, phrased unlike the ask) → expansion usually lifts recall and MRR. This is the case Module 1 was built for; `--crosstext` targets it directly.

- **Single-passage factual questions** → little or no lift; the one query already found the one source. Expansion can't help where there's nothing extra to find.
- **Watch the gate rates.** `relevance = max` across variants means expansion's `answ` rate can only rise or hold (more chances to clear the bar) — but confirm `refus` on negatives stays high. If expansion starts answering questions it should refuse, that's over-expansion (Lesson 4.2), and the table is how you'd catch it.

The decision rule: **enable expansion globally only if `eval-expand` shows a real recall lift on your gold set without hurting the refusal rate.** That's enabling on numbers, not vibes.

> **Under the hood — same harness, honest comparison**
>
> Because both rows come from the identical `run_eval` over the identical gold set and embedder, the only variable is expansion. That's a clean A/B. If you change `query_expand_variants` or toggle `query_expand_hyde` in `.env` and re-run, you're now A/B-testing *those* — the harness turns every knob in Module 4 into something you can justify with a number.

Hands-on

1. Run the comparison on your retrieval gold set, then the cross-text one:

```
uv run maayan eval-expand
uv run maayan eval-expand --crosstext
```

Compare the `no-expand` and `expand` rows. Where is recall@k higher? Is the cross-text lift bigger than the plain one? (It should be.)

2. A/B a knob. Set `QUERY_EXPAND_HYDE=false` in `.env`, re-run `eval-expand --crosstext`, and compare to the HyDE-on numbers. Does HyDE earn its extra call on *your* corpus? Now you're tuning with evidence — exactly the muscle Module 4 builds.

You should now be able to say…

- What `eval-expand` does: A/B the retriever (expansion off vs on) over your gold set, reusing Course 1's harness because the multi-query retriever is a drop-in `Retrieving`.
- That **recall@k** is the lift to watch, expansion helps most on **conceptual/cross-text** questions, and the gate rates guard against over-expansion.
- The rule: enable expansion globally only on a measured recall lift that doesn't hurt refusal.

Next: **4.1 — The knobs, and recipes** — turn what you've measured into settings.

Module 4 — Tuning & playing (making it yours)

Confidently turn the knobs — and even edit the prompts — to fit your corpus, your model, and your taste, without breaking the guarantees.

Lesson 4.1 — The knobs, and recipes

> Module 4, Lesson 1 · ~12 min read + a hands-on. The one question this answers: **what does every new setting do, and what are some good ready-made combinations for different situations?**

Everything in Course 2 is config-driven (the house rule from Course 1, Module 5 — nothing hardcoded). Here's the full panel of new dials and four recipes that combine them well. All live in config.py and .env.example.

The dials

Setting	Default	What it does
`QUERY_EXPAND_ENABLED`	`false`	Master switch for Move 1. Off → plain Course-1 retrieval.
`QUERY_EXPAND_LEXICON`	`true`	Include the free, deterministic lexicon expander (when expand on).
`QUERY_EXPAND_HYDE`	`true`	Add the HyDE hypothetical-passage query (1 model call; needs a backend).
`QUERY_EXPAND_VARIANTS`	`3`	How many LLM reformulations to request (`0` = none; needs a backend).
`QUERY_EXPAND_MAX_QUERIES`	`6`	Hard cap on total queries after dedupe (incl. the original).
`RAG_REASONING_ENABLED`	`false`	Master switch for Move 2 (analyze → synthesize).
`ANSWER_VERIFY_ENABLED`	`false`	Master switch for Move 3 (flag unsupported claims).

Two interactions worth internalizing:

- **`MAX_QUERIES` is your spend cap on width.** Lexicon + variants + HyDE + original can exceed it; the cap (after dedupe) keeps any one ask from ballooning into many retrievals.
- **Expansion with `variants=0` + `hyde=false` = lexicon-only**, which costs **zero model calls** (Lesson 3.1). That's how you get "free" widening even with a backend configured.

Four recipes

Set these in `.env` (CLI flags override per-call when you want to deviate).

1. Cheap & fast — wider net, one generation, minimal extra cost:

```
QUERY_EXPAND_ENABLED=true
QUERY_EXPAND_VARIANTS=2
QUERY_EXPAND_HYDE=false
RAG_REASONING_ENABLED=false
ANSWER_VERIFY_ENABLED=false
```

2. Lexicon-only — zero added model calls; leans entirely on your curated terms:

```
QUERY_EXPAND_ENABLED=true
QUERY_EXPAND_LEXICON=true
QUERY_EXPAND_VARIANTS=0
QUERY_EXPAND_HYDE=false
```

3. Deep & thorough — the full chavrusa, for questions you'll act on or publish:

```
QUERY_EXPAND_ENABLED=true
QUERY_EXPAND_VARIANTS=4
QUERY_EXPAND_HYDE=true
RAG_REASONING_ENABLED=true
ANSWER_VERIFY_ENABLED=true
```

4. Local model (Ollama) — private/offline; keep it lean because small models tire:

```
GENERATION_BACKEND=ollama
OLLAMA_MODEL=qwen2.5:7b-instruct
QUERY_EXPAND_ENABLED=true
QUERY_EXPAND_VARIANTS=2
RAG_REASONING_ENABLED=true     # try it; if study maps are thin, set false
```

A smaller local model writes weaker reformulations and thinner study maps, especially in Hebrew (the same tradeoff Course 1 flagged for the Ollama swap). Measure with `eval-expand` and trust the numbers, not the recipe.

Hands-on

1. Apply a recipe and feel it. Put **Recipe 1** in `.env`, then run a normal `ask` (no flags) and confirm it now expands by default:

```
uv run maayan ask "מה הקשר בין צמצום לבריאת העולם"   # uses .env defaults now
```

2. Override per call. With Recipe 1 still in `.env`, force the full treatment on one question without editing config:

```
uv run maayan ask "..." --reason --verify --show-reasoning
```

Flags beat `.env`. That's your everyday workflow: a sensible default in `.env`, the big guns on demand.

3. Justify it. Before committing a recipe globally, run `eval-expand` (Lesson 3.2) under it. Keep the recipe only if the numbers agree.

You should now be able to say…

- What every new dial does, and the two key interactions (`MAX_QUERIES` as a width cap; `variants=0 + hyde=false` = free lexicon-only expansion).
- Four working recipes and the situation each fits.
- The workflow: a default recipe in `.env`, per-call flag overrides, and `eval-expand` to justify going global.

Next: **4.2 — Editing the prompts (and diagnosing failures)**.

Lesson 4.2 — Editing the prompts (and diagnosing failures)

> Module 4, Lesson 2 · ~14 min read + a hands-on. The one question this answers: **the analyze/synthesis/verify prompts are just strings — how do I safely change them to fit my taste, and how do I recognize and fix the common failure modes?**

The deepest tuning isn't a config number — it's the *prompts*. maayan keeps them injectable, not hardcoded, exactly so you can shape the model's behavior without forking the logic. This lesson shows how, and catalogs what goes wrong.

The prompts are injectable, by design

Every reasoning prompt is a constructor argument with a sensible default. `RAGService` takes `analyze_prompt=` and `verify_prompt=` (defaulting to `ANALYZE_SYSTEM_PROMPT` / `VERIFY_SYSTEM_PROMPT` in generate/rag.py); the expanders take `multi_query_system_prompt=` / `hyde_system_prompt=`. So you override a prompt by *passing a different string* — never by editing library code:

```
rag = RAGService(retriever, backend, score_threshold=0.45,
                 reasoning=True,
                 analyze_prompt=MY_ANALYZE_PROMPT)   # your wording, same
machinery
```

This is the DI house rule (Course 1, Module 5) doing real work: behavior is data you inject, so you can experiment freely and A/B the result with `eval`-style runs, with zero risk to the defaults everyone else uses.

When you edit a prompt, **keep the contract**: the analyze prompt must still say *only the sources, refer by [S#], don't answer yet*; the verify prompt must still emit `OK` or one sentence per line (that's what `parse_unsupported` expects). Change the *style* and *emphasis*, not the *interface*.

The failure catalog

Three things go wrong in practice. Each has a tell and a fix.

1. Over-expansion drowns the signal. Too many/too-broad queries pull in loosely-related passages; RRF's consensus reward then floats generic chunks to the top, and `eval-expand` shows recall flat or

the refusal rate dropping.

- *Tell:* sources drift off-topic; `eval-expand` refusal rate falls on negatives.
- *Fix:* lower `QUERY_EXPAND_VARIANTS`, lower `QUERY_EXPAND_MAX_QUERIES`, or tighten the multi-query prompt to demand *on-topic* rephrasings only.

2. The study map argues instead of mapping. A weak or over-eager model starts answering in Stage 1 — the map editorializes, or asserts connections the sources don't support.

- *Tell:* `--show-reasoning` shows a map that reads like a mini-essay, or claims tensions/links you can't find in the sources.
- *Fix:* strengthen "do NOT answer yet" and "use ONLY what the source says" in the analyze prompt; on a small local model, consider turning reasoning off (Lesson 4.1).

3. The verifier flags everything (or nothing). Some models are trigger-happy citation checkers; others rubber-stamp.

- *Tell:* every answer shows a long ⚠ list (false alarms), or a clearly-overstated answer shows none.
- *Fix:* in the verify prompt, sharpen what "supported" means (e.g. "a claim is supported if the cited source states or directly implies it; minor paraphrase is fine"); or accept that on a weak model, verify is noise and leave it off.

A fourth, quieter one: **forced connections.** Reasoning's superpower — relating sources — is also its temptation; the model may manufacture a link to seem insightful. The study map makes this *visible* (you can read the claimed connection and check it), which is the best defense. Trust the map you can audit over an answer you can't.

> **Under the hood — why flag-only verify is the safe default**
>
> Note we never let verify *rewrite* the answer (Lesson 2.4). A model confident enough to flag a claim is not necessarily right that it's unsupported — so we surface the doubt to you rather than act on it. If you ever wanted auto-correction, that's a *new* feature with its own risks, not a tweak to this one.

Hands-on

1. Inject a custom analyze prompt and compare maps. Try making the map terser:

```
uv run python - <<'PY'
from maayan.config import Settings
from maayan.generate.factory import build_generation_backend
from maayan.retrieve.factory import build_retriever
from maayan.generate.rag import RAGService

TERSE = ("Produce a STUDY MAP: one line per source as '[S#] — claim' (max
12 words each), "
         "then a single line naming the key agreement and the key tension
by [S#]. "
         "Use ONLY the sources. Do not answer the question.")

s = Settings(); backend = build_generation_backend(s)
r = build_retriever(s)
for label, prompt in [("default", None), ("terse", TERSE)]:
    rag = RAGService(r, backend, score_threshold=s.score_threshold,
reasoning=True,
                     **({} if prompt is None else {"analyze_prompt":
prompt}))
    ans = rag.ask("מה הקשר בין צמצום לבריאת העולם")
    print(f"\n=== {label} map ===\n{ans.reasoning}")
PY
```

Read both maps. Which scaffolds a better answer for *your* taste? That judgment, made on your own corpus, is the real tuning.

2. Provoke a failure on purpose. Crank `QUERY_EXPAND_VARIANTS=8` and re-run `eval-expand --crosstext`. Watch whether recall actually keeps rising or whether the refusal rate starts slipping — see over-expansion in your own numbers, then dial back.

You should now be able to say…

- That every reasoning/expansion prompt is **injectable** (a constructor arg), so you tune behavior by passing strings — never editing logic — while keeping each prompt's *contract*.
- The failure catalog and each one's tell + fix: over-expansion, an arguing study map, a mis-calibrated verifier, and forced connections.
- Why flag-only verify is the safe default, and why an auditable study map is your best defense against manufactured connections.

Next: **5.1** — **The five-minute demo** — now show it to someone.

Module 5 — Demoing & explaining (telling the story)

Show this to other people and have it land — scholar, engineer, or skeptic — and know how to carry the pattern forward.

Lesson 5.1 — The five-minute demo

> Module 5, Lesson 1 · ~12 min read + a rehearsal. The one question this answers: **if I have five minutes and someone's attention, exactly what do I type, and what do I point at, to make the upgrade land?**

A demo is a story with a payoff. The story here is "it stopped being a search box and started learning like a chavrusa." Below is an exact script — commands in order, and the one line to say at each beat. Rehearse it until it's muscle memory; nothing kills a demo like fumbling syntax.

Before they arrive (setup)

- Qdrant up, corpus indexed, a working backend. Pick **one conceptual question** you know your corpus answers well, and **one question you know it can't** (for the refusal beat).
- Have the terminal font large. Pre-type nothing — typing live is part of the credibility.

The script (≈5 minutes)

Beat 1 — the baseline (45 sec). Set the "before."

> "Here's the old way — one search, one answer."

```
uv run maayan ask "<your conceptual question>"
```

Point at the **Sources** list and the answer. Say: *"Good, grounded, cited. But it's a lookup — it found passages and reported them."*

Beat 2 — ask better (60 sec). Show retrieval getting smarter.

> "Now let it ask the question several ways and fuse the results."

```
uv run maayan ask "<same question>" --expand
```

Point at a source in the list that **wasn't** there in Beat 1. Say: *"It rephrased the question, even imagined a source passage, and searched with all of them. That source it just found? My original wording missed it."*

Beat 3 — the payoff: it thinks (90 sec). This is the moment.

> "Now watch it study the sources before answering."

```
uv run maayan ask "<same question>" --expand --reason --show-reasoning
```

Read the study map aloud — one source's claim, then "this builds on that, but this is in tension." Say: *"That's not the answer yet — that's it laying out the sugya, the way a chavrusa would. Now the answer is built from that map."* Point at where the answer **connects** sources rather than listing them.

Beat 4 — the connection (30 sec). If the study map surfaced a non-obvious link, dwell on it.

> "Notice it connected [S2] to [S1] — that's the kind of thing you'd capture and teach." This is the "it found a chiddush" beat. It's the most impressive thing the system does; let it breathe.

Beat 5 — the honesty (45 sec). Close on trust, not flash.

> "And it still refuses when it has no source."

```
uv run maayan ask "<a question your corpus can't answer>"
```

Point at the refusal. Say: *"No source, no answer — it never invents a mekor. All that intelligence sits **around** that rule, never through it."*

What to bring out (and what to skip)

- **Bring out:** the *new source* in Beat 2, the *study map* in Beat 3, the *connection* in Beat 4, the *refusal* in Beat 5. Those four beats are the whole story.
- **Skip:** config flags, RRF math, model-call counts. Nobody watching a demo cares; it's noise against the story. (Save it for the engineer in Lesson 5.2.)

- **If a backend call is slow,** narrate it: "it's making a couple of model calls — rephrasing, then studying, then answering." Latency becomes evidence of work, not dead air.

> **Under the hood — why `--show-reasoning` is the star**
>
> The single most persuasive thing you can do is make the *thinking visible*. Anyone can claim an AI "reasons"; almost no one shows you the reasoning as an inspectable artifact you can check against the sources. The study map is your differentiator on screen — lead with it.

Hands-on

Rehearse the full script end to end **twice**, out loud, on your own corpus. Time it. Then do it once for a friend who knows nothing about the project and watch where their eyes light up (it'll be Beat 3 or 4). Adjust which question you open with so those beats hit hardest.

You should now be able to say…

- The five-beat arc: baseline → ask-better → it-thinks → the-connection → the-refusal.
- The four things to *point at*, and the things to *skip* (flags, math, costs).
- Why making the reasoning **visible** (`--show-reasoning`) is the most persuasive move you have.

Next: **5.2 — Explaining it to three audiences** — same system, three framings.

Lesson 5.2 — Explaining it to three audiences

> Module 5, Lesson 2 · ~12 min read + a writing exercise. The one question this answers: **how do I explain what this does to a scholar, an engineer, and a skeptic — bringing out what each cares about, without over-claiming?**

The demo (5.1) shows; this lesson tells. The same system needs three different framings, because three audiences value three different things. Get the framing wrong and a true description still falls flat.

To a scholar (a posek, a mashpia, a talmid chacham)

What they care about: faithfulness to the sources, and whether this respects the way Torah is learned.

The framing: *"It's a tireless chavrusa that has read the whole corpus. You ask, and it pulls the relevant mekoros, lays out how they fit — which builds on which, where there's a tension — and then explains, citing every source. And it will tell you plainly when the texts it has don't answer your question, rather than make something up."*

Bring out: the **study map** (this is the chavrusa move — "it's being mevarer the sugya before answering"), the **citations** on every claim, and the **refusal**. Show that expert connections you capture become part of what it knows (the loop from Course 1).

Never over-claim: it is not a posek; it does not *know* Torah, it *retrieves and relates* what you've indexed; it can still be wrong and must be checked. Say this plainly — a scholar's trust is earned by your honesty about limits, and the system's own refusal behavior backs you up.

To an engineer

What they care about: the architecture, and whether it's sound or hand-wavy.

The framing: *"It's RAG with an intelligence layer: **expand → retrieve → fuse → reason → verify**. Query expansion (multi-query + HyDE + a deterministic lexicon pass) widens retrieval and RRF-fuses the hits; a two-stage generate (analyze the sources into a study map, then synthesize) replaces the single shot; an optional verify pass flags unsupported claims. All of it sits behind two protocols — `Retrieving` and `GenerationBackend` — so the multi-query retriever is a drop-in and the cloud/local model swap is unchanged."*

Bring out: that everything is **off by default and config-gated** (zero risk to baseline); that the **default-deny gate runs before any model call** (enforced in code, not just prompt); that it's **typed, DI'd, and tested with the network/models mocked**; and that the lift is **measured** (`eval-expand`), not asserted.

Never over-claim: it's **not agentic** — there's no retrieve→reason→retrieve loop yet (Lesson 5.3). Say so; engineers respect a clean scope boundary far more than a vague "it reasons."

To a skeptic (a funder, a gabbai, a "isn't this just ChatGPT?" relative)

What they care about: can it be trusted, and what's the catch.

The framing: *"Unlike a chatbot, it can only answer from a specific library of texts, it shows you exactly which passages it used, and — the important part — it refuses when those texts don't cover your question instead of inventing an answer. The recent upgrade made it reason over those sources rather than just find them, but the refusal rule is unchanged."*

Bring out: the **refusal** (do the Beat-5 demo live — nothing convinces a skeptic like watching it decline to answer), and the **citations** (every claim is checkable against a real source).

Never over-claim: don't promise it's always right; promise it's always **grounded and checkable**, and that a human stays in the loop (it flags doubt, it doesn't auto-correct itself). "Trustworthy because it shows its work and knows what it doesn't know" beats "it's smart."

The through-line

Notice the same two facts anchor all three pitches: **it's grounded (citations) and it's honest (refusal).** The intelligence layer is the exciting part, but trust is the *foundation* you sell to every audience. Lead exciting, land on trustworthy.

Hands-on

Write **three one-paragraph pitches** — scholar, engineer, skeptic — for *your* project in *your* words, using the framings above as scaffolding. Then read each aloud and cut every sentence that over-claims. Keep them in your back pocket; you'll give one of these three talks far more often than you'll give the demo.

You should now be able to say…

- The one thing each audience cares about (faithfulness / architecture / trust) and the framing that leads with it.

- What to **bring out** for each (study map / the protocol pipeline / the refusal) and the specific thing to **never over-claim** to each.
- The through-line: grounded + honest is the foundation you sell to everyone.

Next: **5.3 — The horizon: the reusable pattern** — carry this beyond maayan.

Lesson 5.3 — The horizon: the reusable pattern

> Module 5, Lesson 3 · ~12 min read + a design sketch. The one question this answers: **what did I really build that I can reuse anywhere — and what's the honest next step beyond it?**

You set out to make the system "more intelligent, not just a fancy lookup." You did. This final lesson lifts the result off *this* corpus so you can carry it into future projects — and names, squarely, the one big thing deliberately left for next time.

The pattern, abstracted

Strip away the Torah specifics and what you built is a general recipe for turning any retrieval system into a reasoning one:

```
EXPAND   — ask the question several ways (rephrase, hypothesize, inject
domain vocabulary)
RETRIEVE — run each query through your existing search
FUSE     — merge the hit-lists by rank (RRF), keeping an absolute relevance
signal for gating
REASON   — analyze the retrieved evidence into a structured map, THEN
compose from the map
VERIFY   — check the output's claims against the evidence; surface doubt,
don't auto-fix
```

None of this is Torah-specific. The *lexicon* expander is the one domain-flavored piece — and the lesson there generalizes too: **any structured domain knowledge you already have (a glossary, a taxonomy, a synonym list) can deterministically widen retrieval for free.** Wherever you build a RAG next — case law, medicine, a codebase, internal docs — this five-step loop applies, and the three trust rules (ground every claim, refuse without evidence, keep the human in the loop) port unchanged.

That's the real takeaway of Course 2: not five features, but a **repeatable way to add intelligence to retrieval without sacrificing trust.**

What's deliberately not here: agentic multi-hop

There's a ceiling we *didn't* break, on purpose. Every answer still does **one** retrieval pass (now widened, but one). It cannot notice mid-answer that "these sources cover X but the question also

needs Y" and **go look again**. That loop — retrieve → reason → *decide more is needed* → retrieve again → synthesize — is **agentic multi-hop**, and it's the natural next rung.

Why it was left out is itself worth understanding. Multi-hop needs the model to *act*, not just emit text: to decide "I need more" and issue a follow-up query. That requires a richer `GenerationBackend` contract — structured output or tool-calling — instead of today's clean string-in/string-out. Adding it touches the one protocol we kept stable all course. So it's not a tweak; it's a deliberate next project, with its own design and its own risks (loops that don't terminate, runaway cost). Knowing *why* a feature is out of scope is as much engineering maturity as knowing how to build one.

The protocols you have are already the seam where it'll attach: `Retrieving` (you'd call it in a loop) and `GenerationBackend` (you'd extend it to return a decision, not just prose). The system was built to grow here.

Honest limits to carry forward

Say these out loud so you never oversell:

- It **retrieves and relates**; it does not *know*. A missing source is a missing answer.
- Reasoning can **manufacture connections**; the study map makes them auditable, but a human must still audit.
- Verify **flags**, it doesn't **fix**; and on a weak model it can mis-flag.
- More intelligence costs **more model calls** (up to 5); spend them where they earn it.

These aren't apologies — they're the boundary that makes the trustworthy core trustworthy.

Hands-on

Sketch the next rung. On paper, design a minimal agentic loop for maayan:

1. After the study map, what *question* would the model ask itself to decide "do I have enough?"
2. What would the follow-up query be, and how would you feed its results back (RRF again?)?
3. What would you add to `GenerationBackend` so it can return *"need more: "* vs. *"ready to answer"* — and where would you cap the number of hops?

You don't have to build it. Sketching it proves you understand both the pattern you completed and the seam where it extends — which is exactly the understanding this course set out to give you.

You should now be able to say…

- The reusable, domain-agnostic pattern: **expand → retrieve → fuse → reason → verify**, plus the three portable trust rules.
- What agentic **multi-hop** is, why it's the next rung, and *why* it was deliberately scoped out (it needs a richer backend contract — the one protocol kept stable).
- The honest limits of what you built — and that naming them is what keeps the core trustworthy.

End of Course 2. You can now explain, run, tune, demo, and extend the intelligence layer — and carry its pattern into whatever you build next. The wellspring runs deeper.